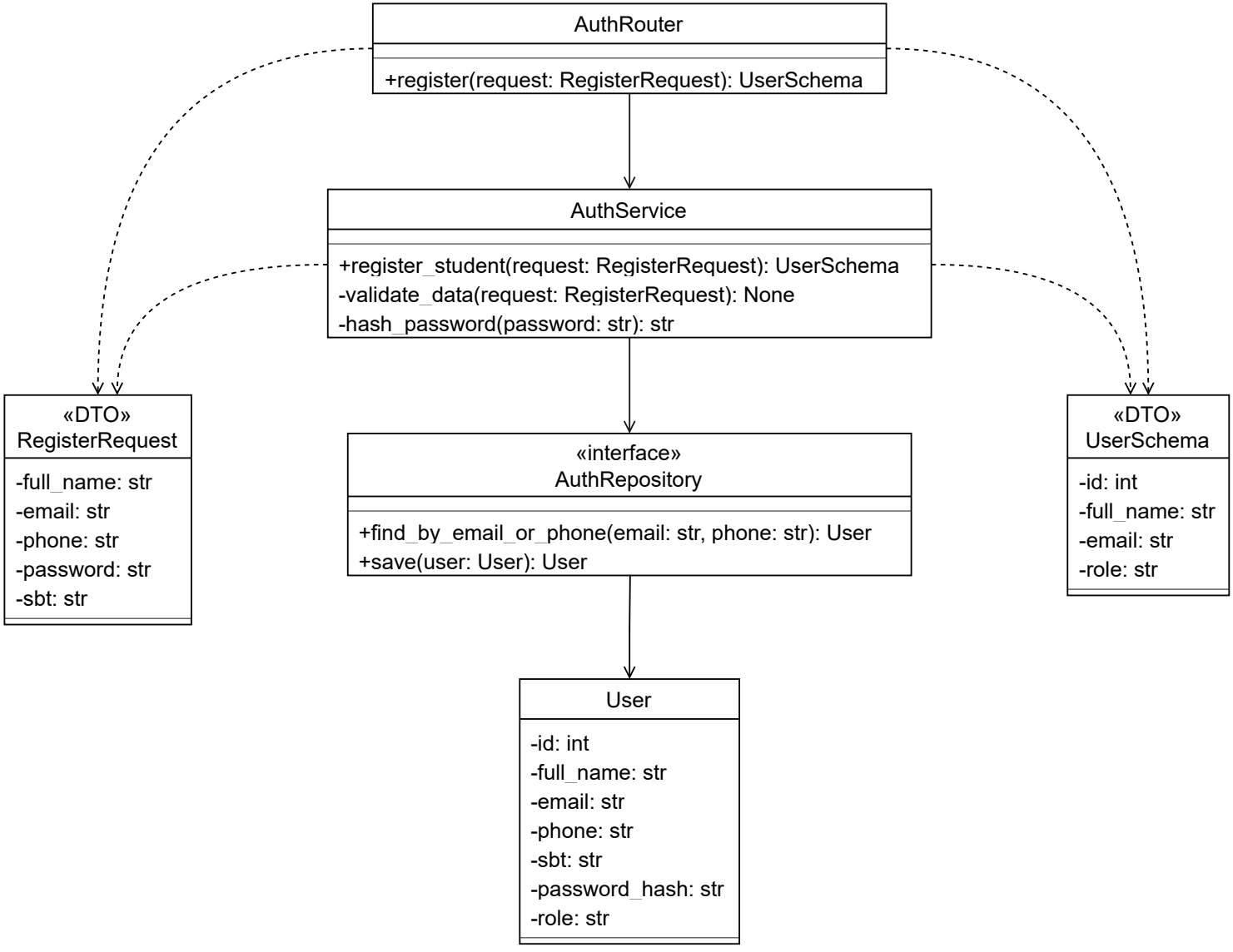
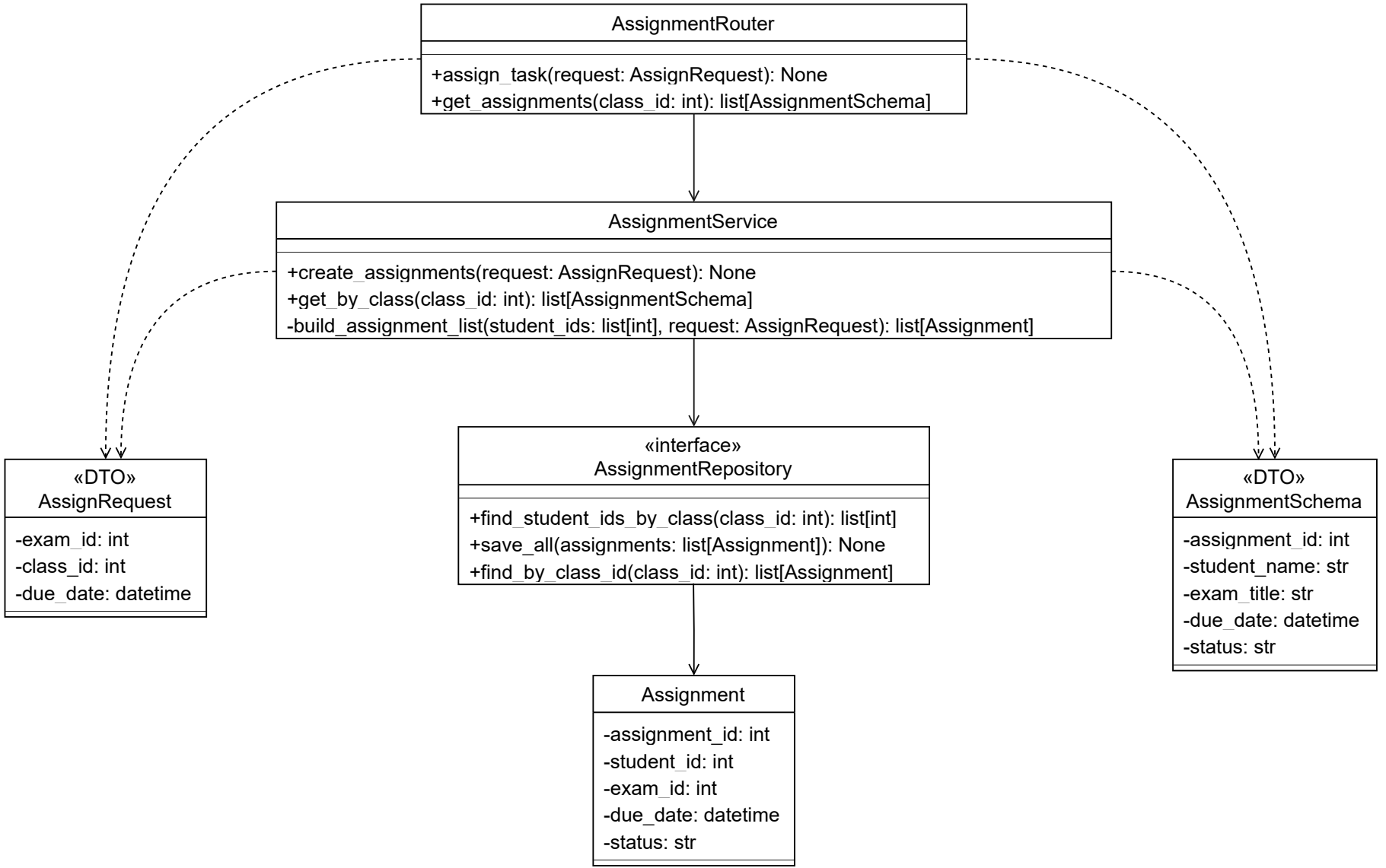
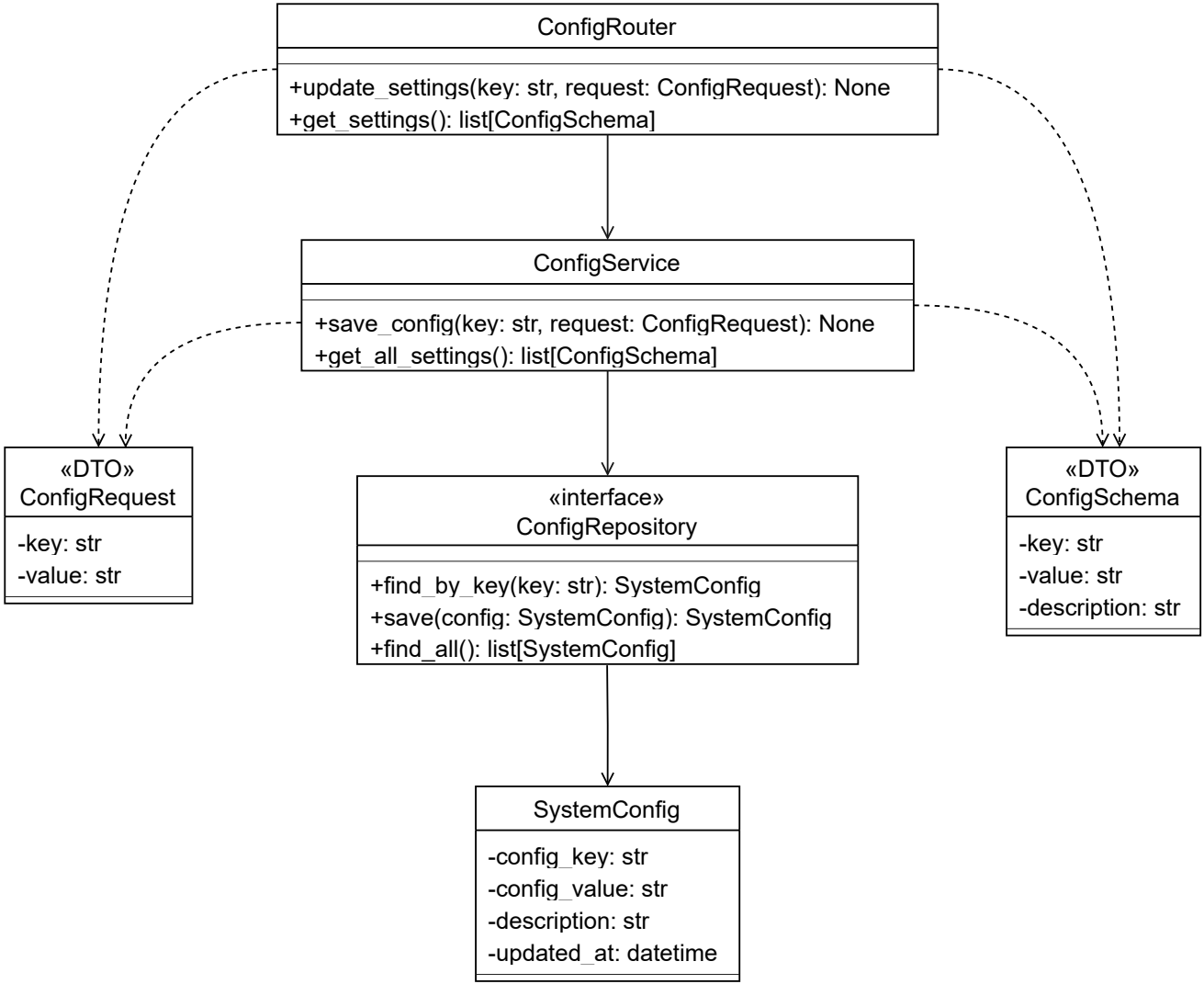
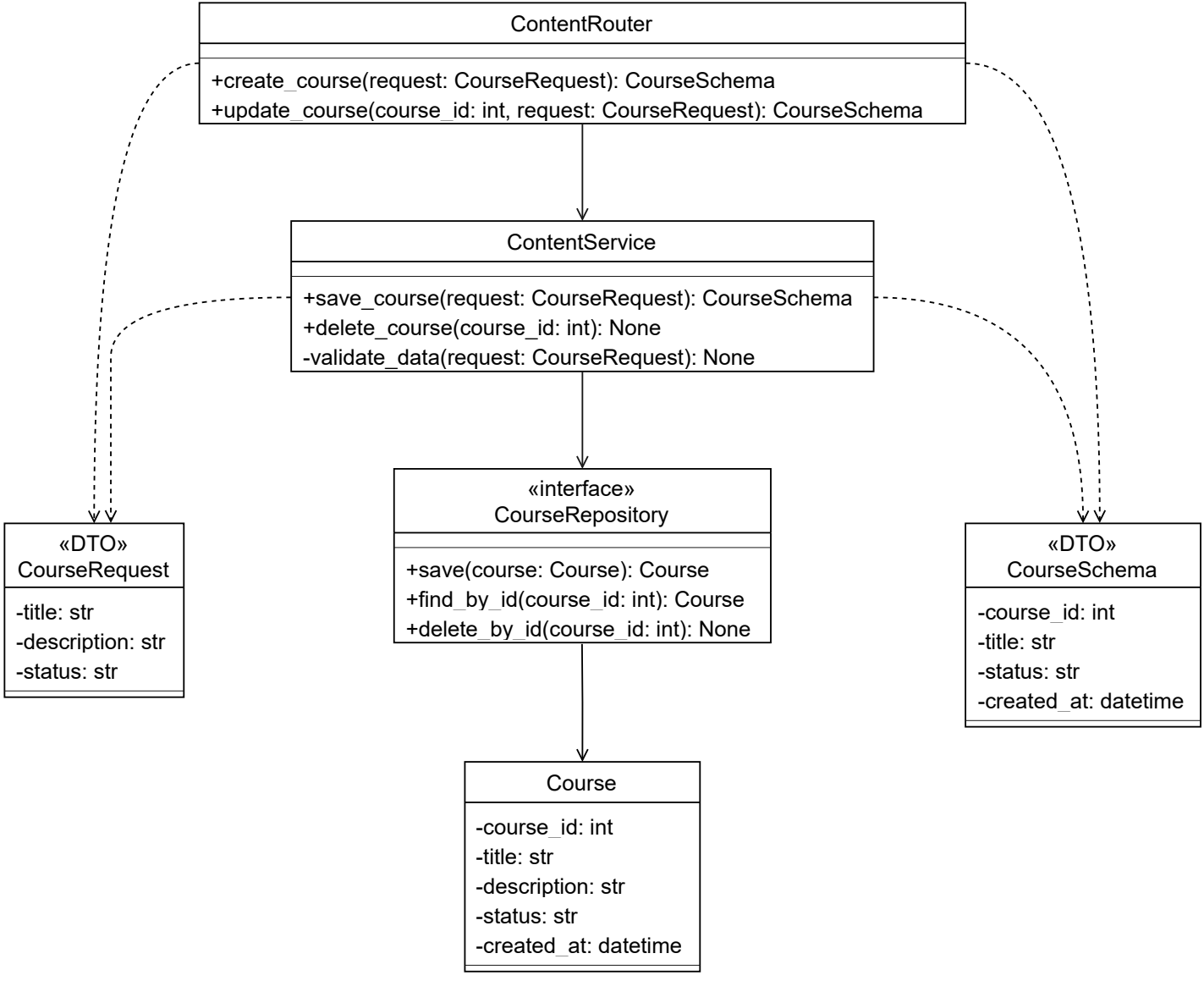
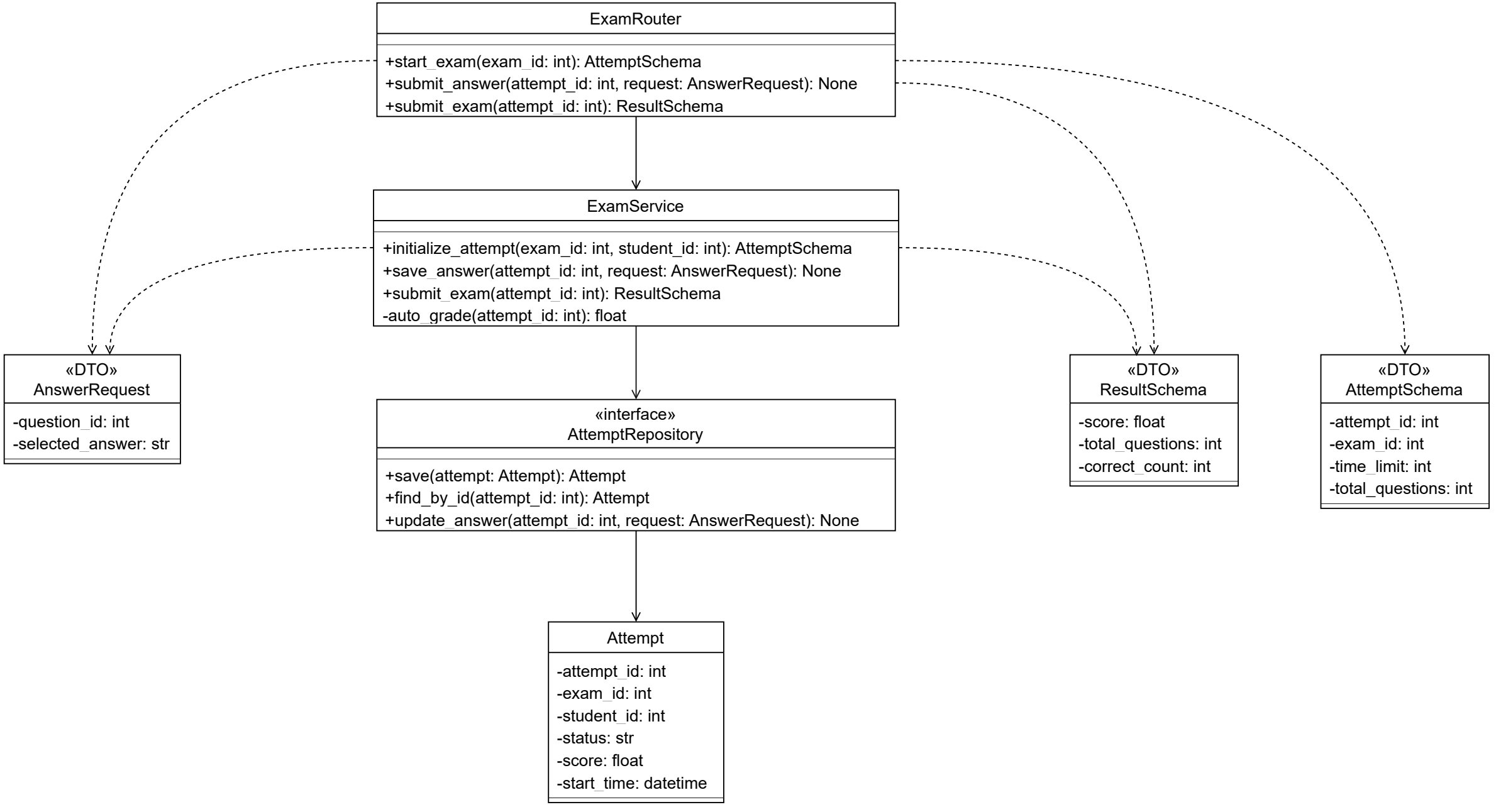
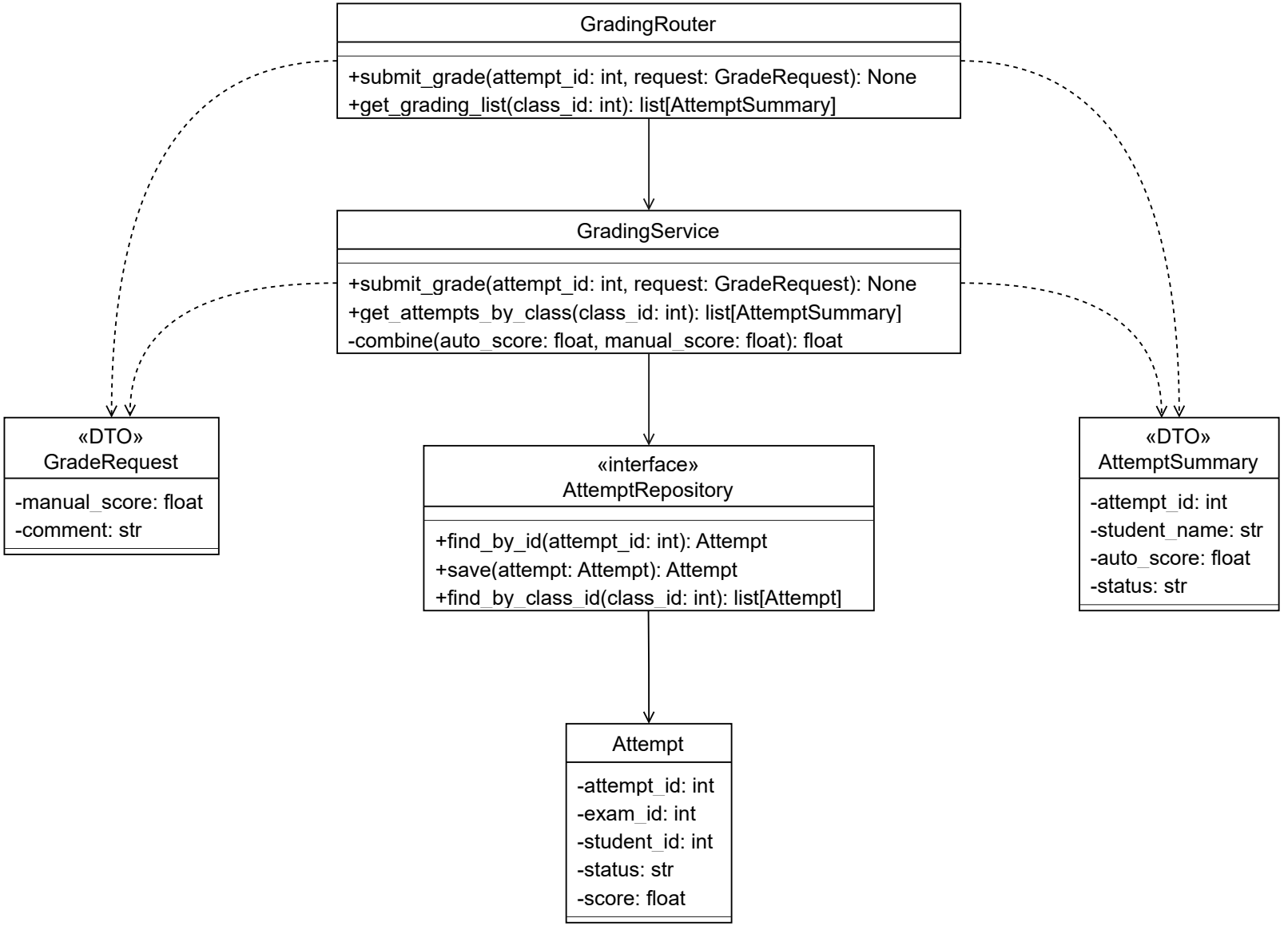

AuthRouter
+register(request: RegisterRequest): UserSchema
AuthService
+register_student(request: RegisterRequest): UserSchema
-validate_data(request: RegisterRequest): None
-hash_password(password: str): str
«DTO»
RegisterRequest
-full_name: str
-email: str
-phone: str
-password: str
-sbt: str
«interface»
AuthRepository
+find_by_email_or_phone(email: str, phone: str): User
+save(user: User): User
«DTO»
UserSchema
-id: int
-full_name: str
-email: str
-role: str
User
-id: int
-full_name: str
-email: str
-phone: str
-sbt: str
-password_hash: str
-role: str

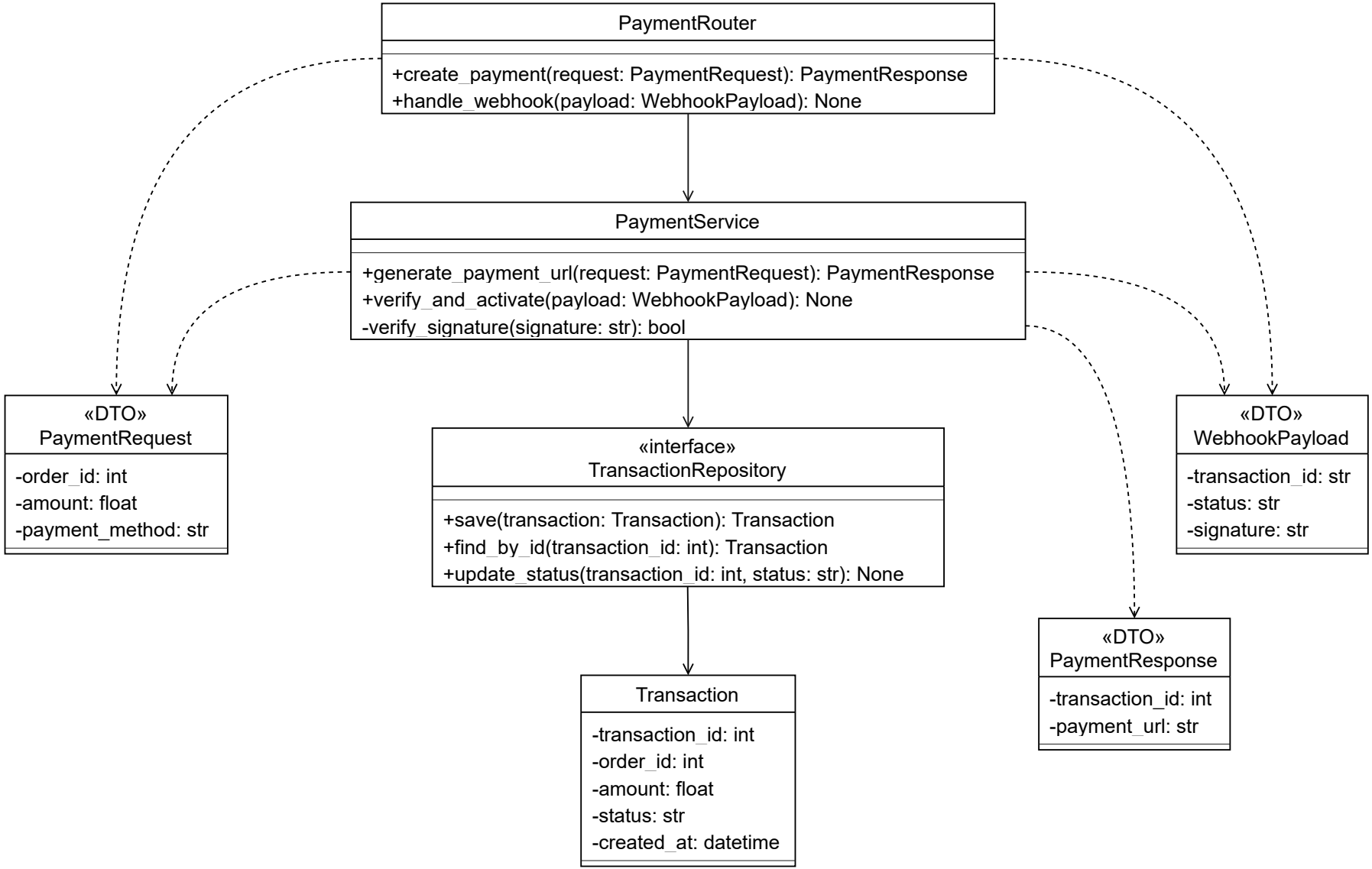

AssignmentRouter
+assign_task(request: AssignRequest): None
+get_assignments(class_id: int): list[AssignmentSchema]
AssignmentService
+create_assignments(request: AssignRequest): None
+get_by_class(class_id: int): list[AssignmentSchema]
-build_assignment_list(student_ids: list[int], request: AssignRequest): list[Assignment]
«DTO»
AssignRequest
-exam_id: int
-class_id: int
-due_date: datetime
«interface»
AssignmentRepository
+find_student_ids_by_class(class_id: int): list[int]
+save_all(assignments: list[Assignment]): None
+find_by_class_id(class_id: int): list[Assignment]
«DTO»
AssignmentSchema
-assignment_id: int
-student_name: str
-exam_title: str
-due_date: datetime
-status: str
Assignment
-assignment_id: int
-student_id: int
-exam_id: int
-due_date: datetime
-status: str

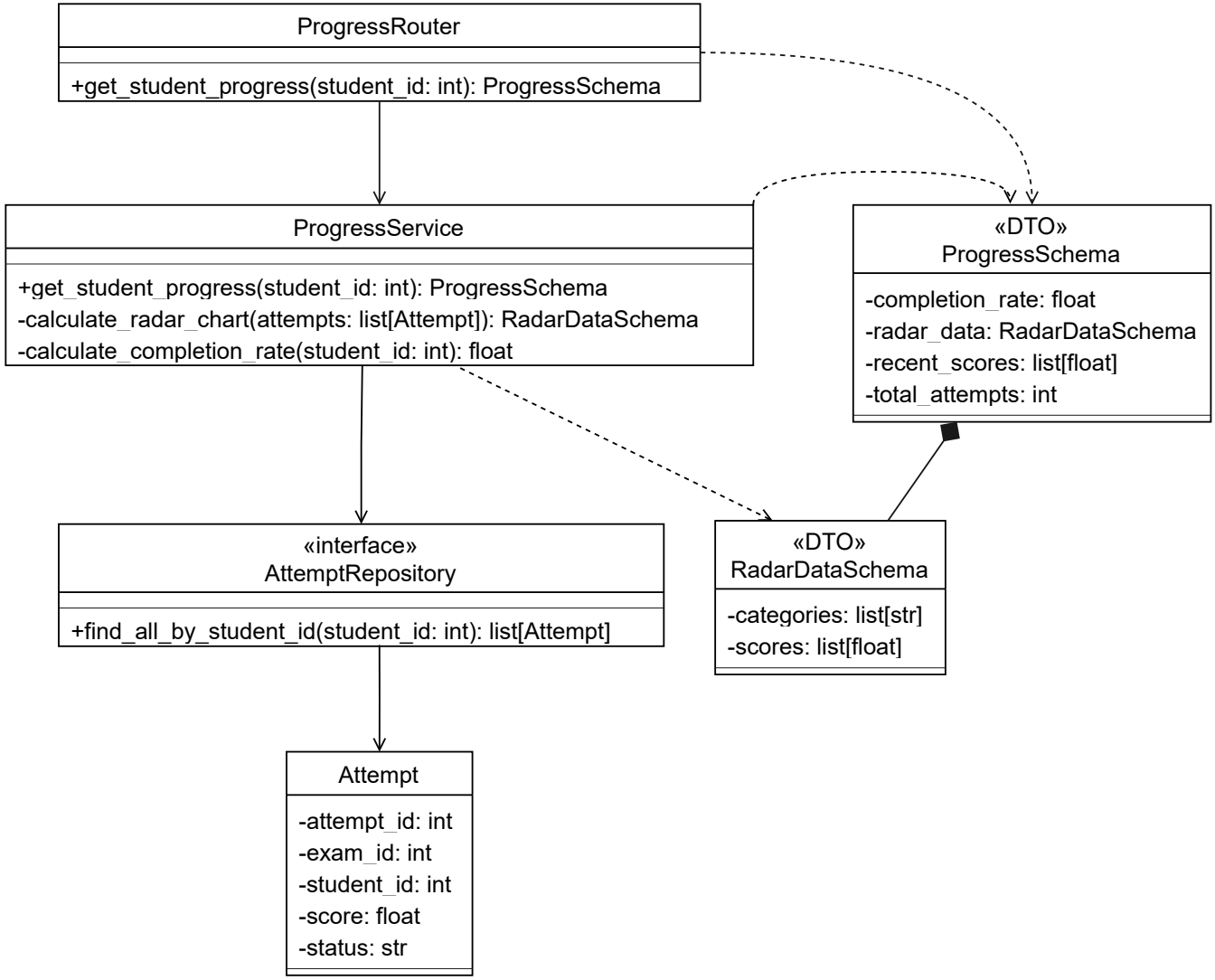

ConfigRouter
+update_settings(key: str, request: ConfigRequest): None
+get_settings(): list[ConfigSchema]
ConfigService
+save_config(key: str, request: ConfigRequest): None
+get_all_settings(): list[ConfigSchema]
«DTO»
ConfigRequest
-key: str
-value: str
«interface»
ConfigRepository
+find_by_key(key: str): SystemConfig
+save(config: SystemConfig): SystemConfig
+find_all(): list[SystemConfig]
«DTO»
ConfigSchema
-key: str
-value: str
-description: str
SystemConfig
-config_key: str
-config_value: str
-description: str
-updated_at: datetime

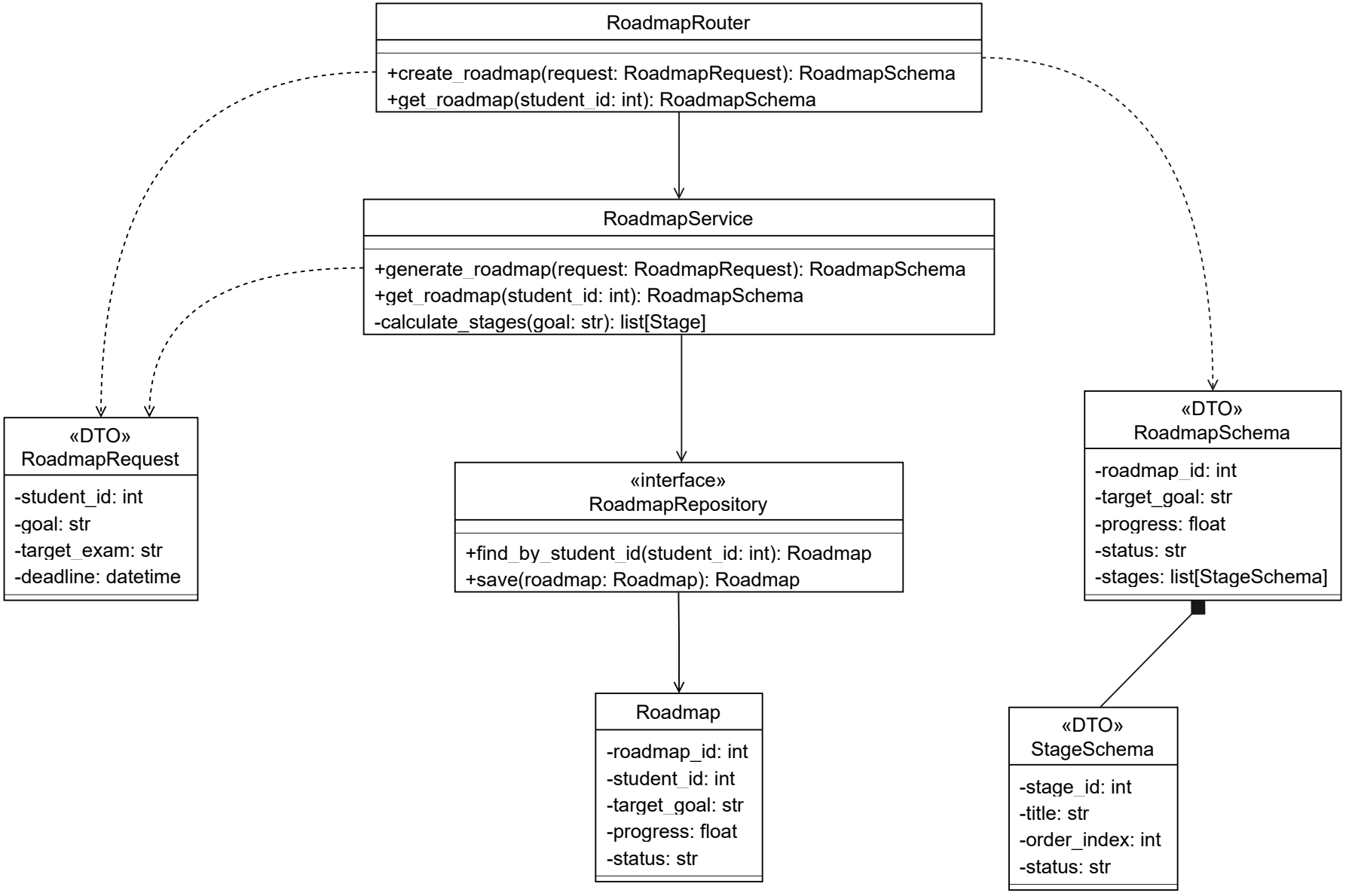

ContentRouter
+create_course(request: CourseRequest): CourseSchema
+update_course(course_id: int, request: CourseRequest): CourseSchema
ContentService
+save_course(request: CourseRequest): CourseSchema
+delete_course(course_id: int): None
-validate_data(request: CourseRequest): None
«interface»
CourseRepository
+save(course: Course): Course
+find_by_id(course_id: int): Course
+delete_by_id(course_id: int): None
«DTO»
CourseRequest
-title: str
-description: str
-status: str
«DTO»
CourseSchema
-course_id: int
-title: str
-status: str
-created_at: datetime
Course
-course_id: int
-title: str
-description: str
-status: str
-created_at: datetime

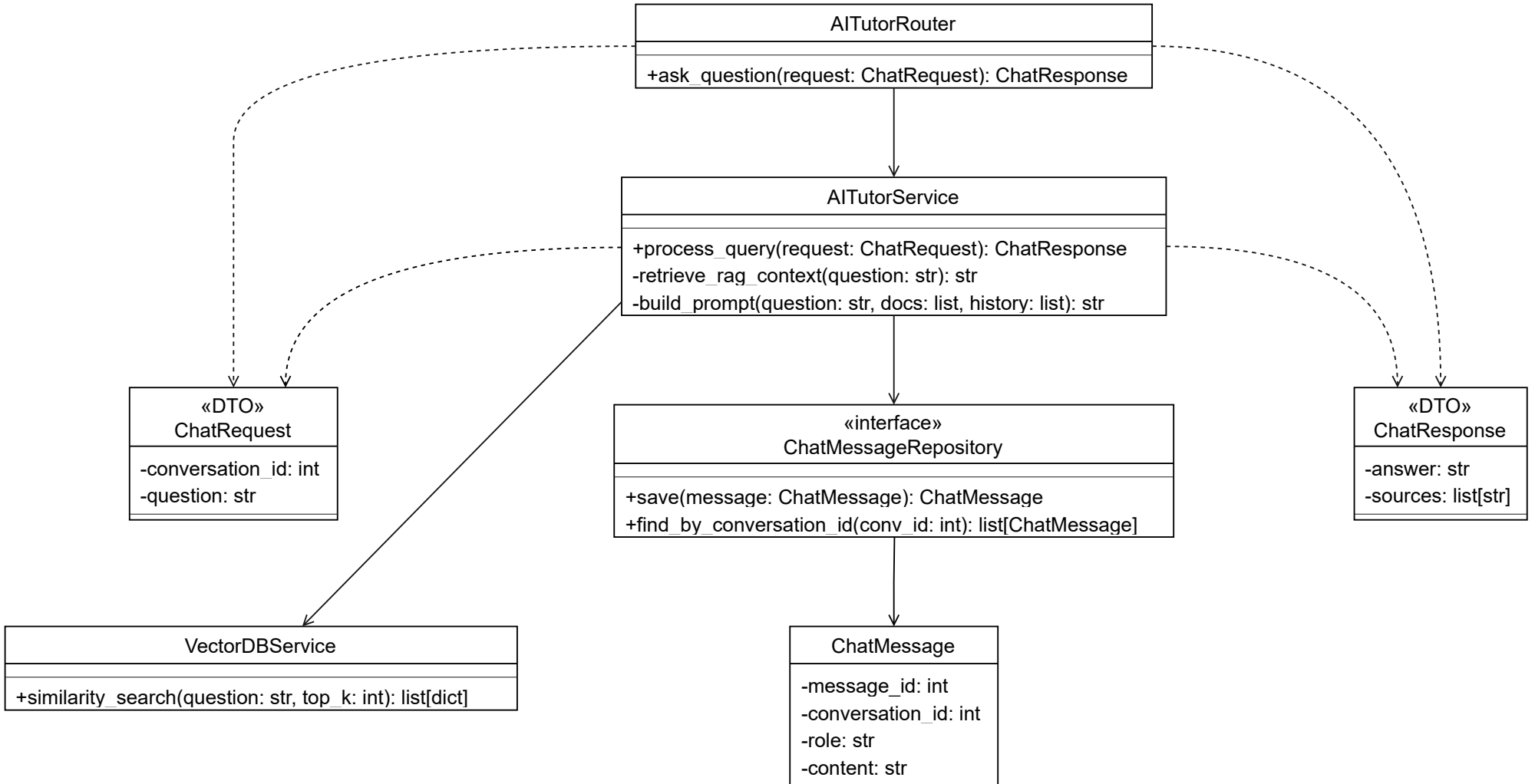

ExamRouter
+start_exam(exam_id: int): AttemptSchema
+submit_answer(attempt_id: int, request: AnswerRequest): None
+submit_exam(attempt_id: int): ResultSchema
ExamService
+initialize_attempt(exam_id: int, student_id: int): AttemptSchema
+save_answer(attempt_id: int, request: AnswerRequest): None
+submit_exam(attempt_id: int): ResultSchema
-auto_grade(attempt_id: int): float
«DTO»
AnswerRequest
-question_id: int
-selected_answer: str
«interface»
AttemptRepository
+save(attempt: Attempt): Attempt
+find_by_id(attempt_id: int): Attempt
+update_answer(attempt_id: int, request: AnswerRequest): None
«DTO»
ResultSchema
-score: float
-total_questions: int
-correct_count: int
«DTO»
AttemptSchema
-attempt_id: int
-exam_id: int
-time_limit: int
-total_questions: int
Attempt
-attempt_id: int
-exam_id: int
-student_id: int
-status: str
-score: float
-start_time: datetime


GradingRouter
+submit_grade(attempt_id: int, request: GradeRequest): None
+get_grading_list(class_id: int): list[AttemptSummary]
GradingService
+submit_grade(attempt_id: int, request: GradeRequest): None
+get_attempts_by_class(class_id: int): list[AttemptSummary]
-combine(auto_score: float, manual_score: float): float
«DTO»
GradeRequest
-manual_score: float
-comment: str
«interface»
AttemptRepository
+find_by_id(attempt_id: int): Attempt
+save(attempt: Attempt): Attempt
+find_by_class_id(class_id: int): list[Attempt]
«DTO»
AttemptSummary
-attempt_id: int
-student_name: str
-auto_score: float
-status: str
Attempt
-attempt_id: int
-exam_id: int
-student_id: int
-status: str
-score: float


PaymentRouter
+create_payment(request: PaymentRequest): PaymentResponse
+handle_webhook(payload: WebhookPayload): None
PaymentService
+generate_payment_url(request: PaymentRequest): PaymentResponse
+verify_and_activate(payload: WebhookPayload): None
-verify_signature(signature: str): bool
«DTO»
PaymentRequest
-order_id: int
-amount: float
-payment_method: str
«interface»
TransactionRepository
+save(transaction: Transaction): Transaction
+find_by_id(transaction_id: int): Transaction
+update_status(transaction_id: int, status: str): None
«DTO»
WebhookPayload
-transaction_id: str
-status: str
-signature: str
«DTO»
PaymentResponse
-transaction_id: int
-payment_url: str
Transaction
-transaction_id: int
-order_id: int
-amount: float
-status: str
-created_at: datetime


ProgressRouter
+get_student_progress(student_id: int): ProgressSchema
ProgressService
+get_student_progress(student_id: int): ProgressSchema
-calculate_radar_chart(attempts: list[Attempt]): RadarDataSchema
-calculate_completion_rate(student_id: int): float
«DTO»
ProgressSchema
-completion_rate: float
-radar_data: RadarDataSchema
-recent_scores: list[float]
-total_attempts: int
«interface»
AttemptRepository
+find_all_by_student_id(student_id: int): list[Attempt]
«DTO»
RadarDataSchema
-categories: list[str]
-scores: list[float]
Attempt
-attempt_id: int
-exam_id: int
-student_id: int
-score: float
-status: str


RoadmapRouter
+create_roadmap(request: RoadmapRequest): RoadmapSchema
+get_roadmap(student_id: int): RoadmapSchema
RoadmapService
+generate_roadmap(request: RoadmapRequest): RoadmapSchema
+get_roadmap(student_id: int): RoadmapSchema
-calculate_stages(goal: str): list[Stage]
«DTO»
RoadmapRequest
-student_id: int
-goal: str
-target_exam: str
-deadline: datetime
«interface»
RoadmapRepository
+find_by_student_id(student_id: int): Roadmap
+save(roadmap: Roadmap): Roadmap
«DTO»
RoadmapSchema
-roadmap_id: int
-target_goal: str
-progress: float
-status: str
-stages: list[StageSchema]
Roadmap
-roadmap_id: int
-student_id: int
-target_goal: str
-progress: float
-status: str
«DTO»
StageSchema
-stage_id: int
-title: str
-order_index: int
-status: str


AITutorRouter
+ask_question(request: ChatRequest): ChatResponse
AITutorService
+process_query(request: ChatRequest): ChatResponse
-retrieve_rag_context(question: str): str
-build_prompt(question: str, docs: list, history: list): str
«DTO»
ChatRequest
-conversation_id: int
-question: str
«interface»
ChatMessageRepository
+save(message: ChatMessage): ChatMessage
+find_by_conversation_id(conv_id: int): list[ChatMessage]
«DTO»
ChatResponse
-answer: str
-sources: list[str]
VectorDBService
+similarity_search(question: str, top_k: int): list[dict]
ChatMessage
-message_id: int
-conversation_id: int
-role: str
-content: str